

Assignment 2

Course Title: Parallel and Distributed Computing (CS-426L)

Title Of Task

" Implementation of Java Data Structures, Object-Oriented Programming, Multithreading, Parallel Processing, and Performance Benchmarking"

SUBMITTED BY:

Name: Ibtehaj Ali Mirza

Student ID: 247200

Semester: 5th

Section: BS-CYS-F-24-**A**

SUBMITTED TO:

Instructor Name: Mr. Qaiser Ali

Date of Submission: 25 October, 2026

AIR UNIVERSITY, KHARIAN CAMPUS

DEPARTMENT OF COMPUTER SCIENCE

Github Repo Link: https://github.com/ibtehajmirza/Parallel-and-Distributed-Computing

Question 1: Implement a Java trading event dispatcher using inheritance and polymorphism.

Code:

TradeEvent.java

```
public class TradeEvent {
    protected String senderId;
    protected long timestamp;
    protected double basePrice;

    public TradeEvent(String senderId, long timestamp, double basePrice) {
        this.senderId = senderId;
        this.timestamp = timestamp;
        this.basePrice = basePrice;
    }

    public void dispatch() {
        System.out.println("[TradeEvent] Generic Dispatch -> Sender: " + senderId
                + " | Timestamp: " + timestamp
                + " | Base Price: $" + String.format("%.2f", basePrice));
    }
}
```

FIXOrderEvent.java

```
public class FIXOrderEvent extends TradeEvent {
    private String clOrdId;

    public FIXOrderEvent(String senderId, long timestamp, double basePrice,
String clOrdId) {
        super(senderId, timestamp, basePrice);
        this.clOrdId = clOrdId;
    }

    @Override
    public void dispatch() {
        double fee = basePrice * 0.0015;
        double totalCost = basePrice + fee;
        System.out.println("[FIXOrderEvent] Executing Stock Purchase:");
        System.out.println("  Client Order ID: " + clOrdId);
```

```
        System.out.println("  Sender ID: " + senderId);
        System.out.println("  Base Price: $" + String.format("%.2f", basePrice));
        System.out.println("  Transaction Fee (0.15%): $" + String.format("%.4f",
fee));
        System.out.println("  Total Execution Cost: $" + String.format("%.2f",
totalCost));
    }
}
```

ReutersQuoteEvent.java

```
public class ReutersQuoteEvent extends TradeEvent {
    private double bidPrice;
    private double askPrice;

    public ReutersQuoteEvent(String senderId, long timestamp, double basePrice,
double bidPrice, double askPrice) {
        super(senderId, timestamp, basePrice);
        this.bidPrice = bidPrice;
        this.askPrice = askPrice;
    }

    @Override
    public void dispatch() {
        double spread = askPrice - bidPrice;
        System.out.println("[ReutersQuoteEvent] Market Analytics:");
        System.out.println("  Feed Source: " + senderId);
        System.out.println("  Bid Price: $" + String.format("%.2f", bidPrice));
        System.out.println("  Ask Price: $" + String.format("%.2f", askPrice));
        System.out.println("  Bid-Ask Spread: $" + String.format("%.4f",
spread));
    }
}
```

TradingEngineApp.java

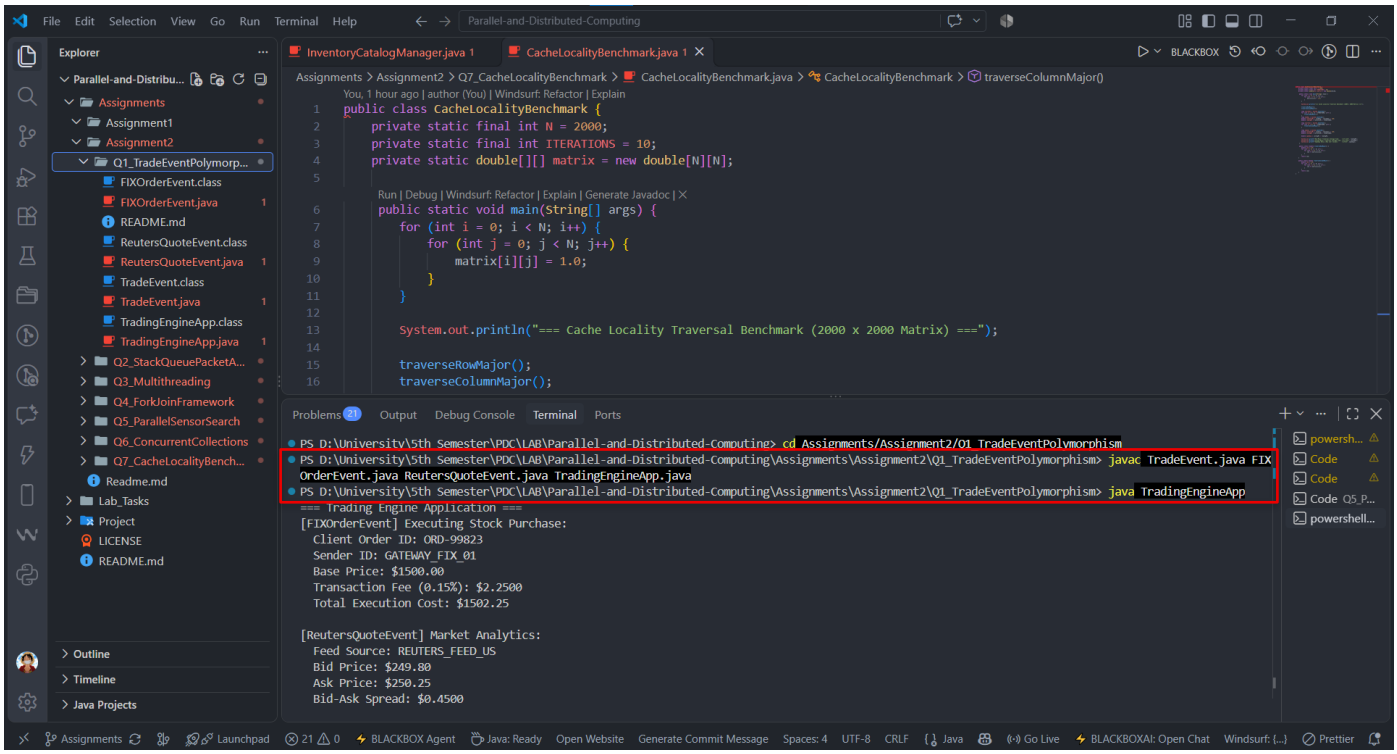
```
public class TradingEngineApp {
    public static void main(String[] args) {
        System.out.println("=== Trading Engine Application ===");

        long now = System.currentTimeMillis();

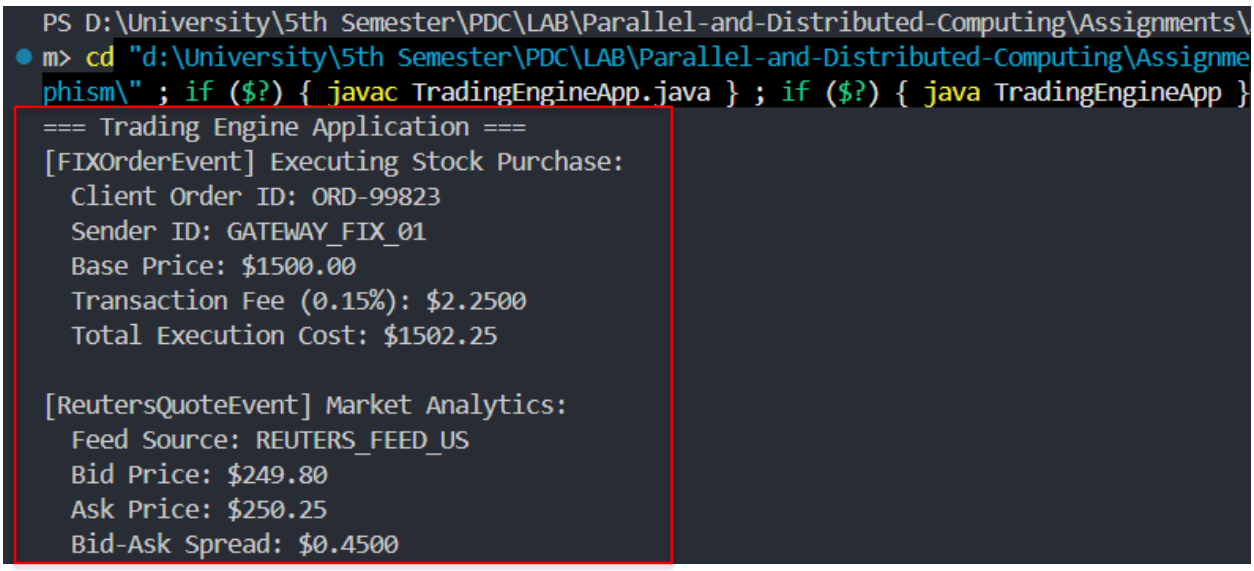
        TradeEvent event1 = new FIXOrderEvent("GATEWAY_FIX_01", now, 1500.00,
"ORD-99823");
        TradeEvent event2 = new ReutersQuoteEvent("REUTERS_FEED_US", now, 250.00,
249.80, 250.25);
```

```
        event1.dispatch();
        System.out.println();
        event2.dispatch();
    }
}
```

Execution/Compilation Commands:



Output:



Explanation:

First, I created the TradeEvent superclass with protected variables for the sender ID, timestamp, and base price, along with a constructor to initialize them and a general dispatch() method. Then, I created the FIXOrderEvent and ReutersQuoteEvent subclasses, which inherit the common data from the parent class and override the dispatch() method to perform their specific tasks. The FIX order class calculates the 0.15% transaction fee and adds it to the base price to find the total execution cost, while the Reuters quote class calculates the bid-ask spread by subtracting the bid price from the ask price. Finally, in TradingEngineApp, I created objects of both subclasses using TradeEvent reference variables and called their dispatch() methods. Java automatically executes the appropriate overridden method for each object, demonstrating inheritance, method overriding, and runtime polymorphism.

Question 2: Develop a Java palindrome checker using custom stacks and queues.

Code:

TransactionStack.java

```
public class TransactionStack {
    private char[] data;
    private int top;

    public TransactionStack(int capacity) {
        data = new char[capacity];
        top = -1;
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public boolean isFull() {
        return top == data.length - 1;
    }

    public void push(char c) {
        if (!isFull()) {
            data[++top] = c;
        } else {
            System.out.println("Stack is full. Cannot push " + c + " onto the
stack.");
        }
    }
```

```
    public char pop() {
        if (!isEmpty()) {
            return data[top--];
        } else {
            System.out.println("Stack is empty. Cannot pop from the stack.");
            return '\0';
        }
    }

    public char peek() {
        if (!isEmpty()) {
            return data[top];
        } else {
            System.out.println("Stack is empty. Cannot peek.");
            return '\0'; // Return null character if stack is empty
        }
    }
}
```

RequestQueue.java

```
public class RequestQueue {
    private char[] data;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public RequestQueue(int capacity) {
        this.capacity = capacity;
        data = new char[capacity];
        front = 0;
        rear = 0;
        size = 0;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public boolean isFull() {
        return size == capacity;
    }
```

```
    public void enqueue(char c) {
        if (!isFull()) {
            data[rear] = c;
            rear = (rear + 1) % capacity;
            size++;
        } else {
            System.out.println("Queue is full. Cannot enqueue " + c + " into the
queue.");
        }
    }

    public char dequeue() {
        if (!isEmpty()) {
            char item = data[front];
            front = (front + 1) % capacity;
            size--;
            return item;
        } else {
            System.out.println("Queue is empty. Cannot dequeue from the queue.");
            return '\0'; // Return null character if queue is empty
        }
    }

}
```

PacketSymmetryAuditor.java

```
public class PacketSymmetryAuditor {
    public static void main(String[] args) {
        String[] testPackets = {"RADAR", "GATEWAY", "1001001", "ABCDBCA"};
        System.out.println("=== Zero-Trust Packet Symmetry Auditor ===\n");

        for (String packet : testPackets) {
            verifyPacket(packet);
        }
    }

    public static void verifyPacket(String packet) {
        int len = packet.length();
        TransactionStack stack = new TransactionStack(len);
        RequestQueue queue = new RequestQueue(len);

        for (int i = 0; i < len; i++) {
            char ch = packet.charAt(i);
```

```
            if (!queue.isFull()) {
                queue.enqueue(ch);
            }

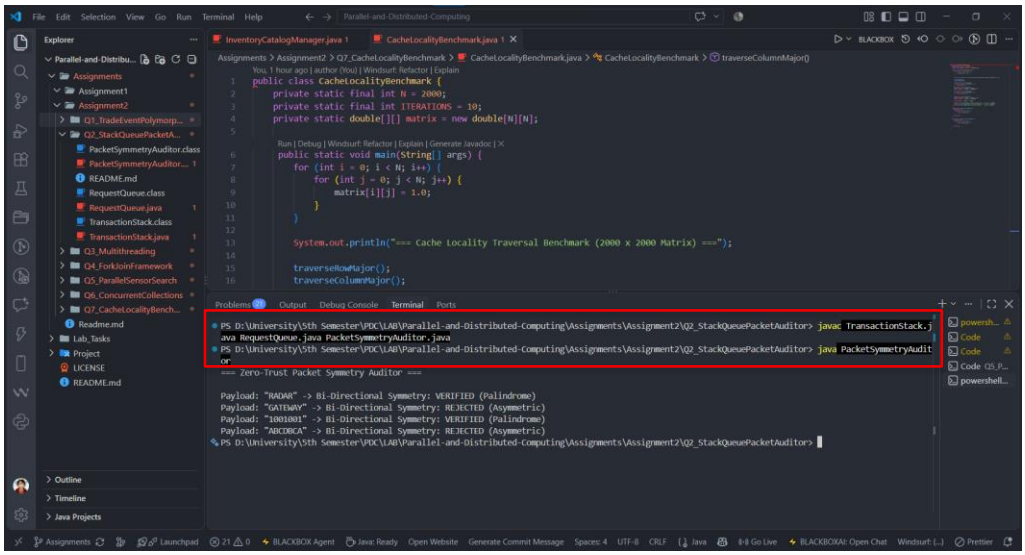
            if (!stack.isFull()) {
                stack.push(ch);
            }
        }

        boolean isSymmetric = true;
        while (!stack.isEmpty() && !queue.isEmpty()) {
            char fromStack = stack.pop();
            char fromQueue = queue.dequeue();
            if (fromStack != fromQueue) {
                isSymmetric = false;
                break;
            }
        }

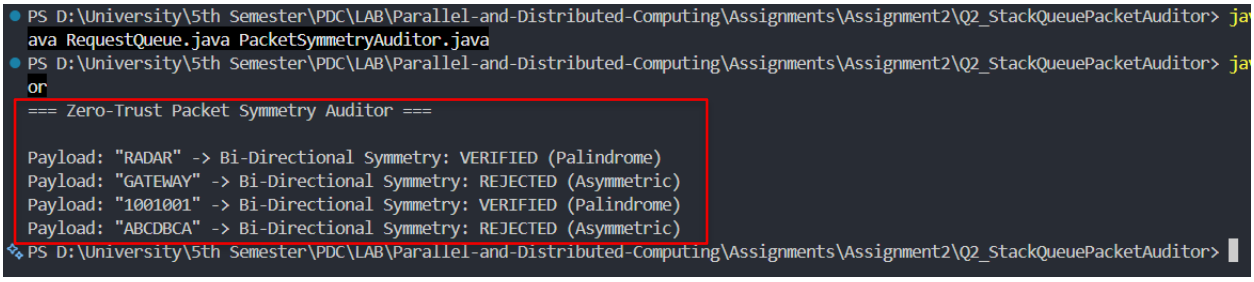
        System.out.println("Payload: \"" + packet + "\" -> Bi-Directional
Symmetry: "
                + (isSymmetric ? "VERIFIED (Palindrome)" : "REJECTED
(Asymmetric)"));

    }
}
```

Execution/Compilation Commands:



Output:



Explanation:

First, I created the TransactionStack class using a character array and a top variable to implement a LIFO stack, where the last character added is removed first. Then, I created the RequestQueue class using a character array and the front, rear, and size variables to implement a FIFO queue, where the first character added is removed first. Both classes include methods to insert and remove characters while checking whether the stack or queue is empty or full. In PacketSymmetryAuditor, I tested four packet strings by inserting their characters into both the stack and queue. The program then compares the characters removed from the stack and queue one by one. If all compared characters match, the packet is marked as a verified palindrome; otherwise, it is rejected as asymmetric. This demonstrates how custom stack and queue data structures can be used to check the symmetry of packet payloads.

Question 3: Demonstrate multithreading using Java Thread and Runnable.

Code:

LogProcessorTask.java

```
public class LogProcessorTask implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i <= 5; i++) {
            System.out.println("[" + Thread.currentThread().getName() + "]
Iteration " + i + ": Processing batch event logs...");

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println("LogProcessor interrupted");
            }
        }
        System.out.println("[" + Thread.currentThread().getName() + "]
Completed.");
    }
```

```
}
```

TelemetryThread.java

```
public class TelemetryThread extends Thread {
    public TelemetryThread(String name) {
        super(name);
    }

    @Override
    public void run() {
        for (int i = 0; i <= 5; i++) {
            System.out.println("[" + getName() + "] Iteration " + i + ": Health
Check -> CPU: OK | Memory: OK");

            try {
                Thread.sleep(800);
            } catch (InterruptedException e) {
                System.out.println("TelemetryThread interrupted");
            }
        }
        System.out.println("[" + getName() + "] Completed.");
    }
}
```

AsyncWebEngineApp.java

```
public class AsyncWebEngineApp {
    public static void main(String[] args) {
        System.out.println("Initial thread: " +
Thread.currentThread().getName());

        TelemetryThread telemetry = new TelemetryThread("Telemetry-Worker");
        Thread logWorker = new Thread(new LogProcessorTask(), "Log-Worker");

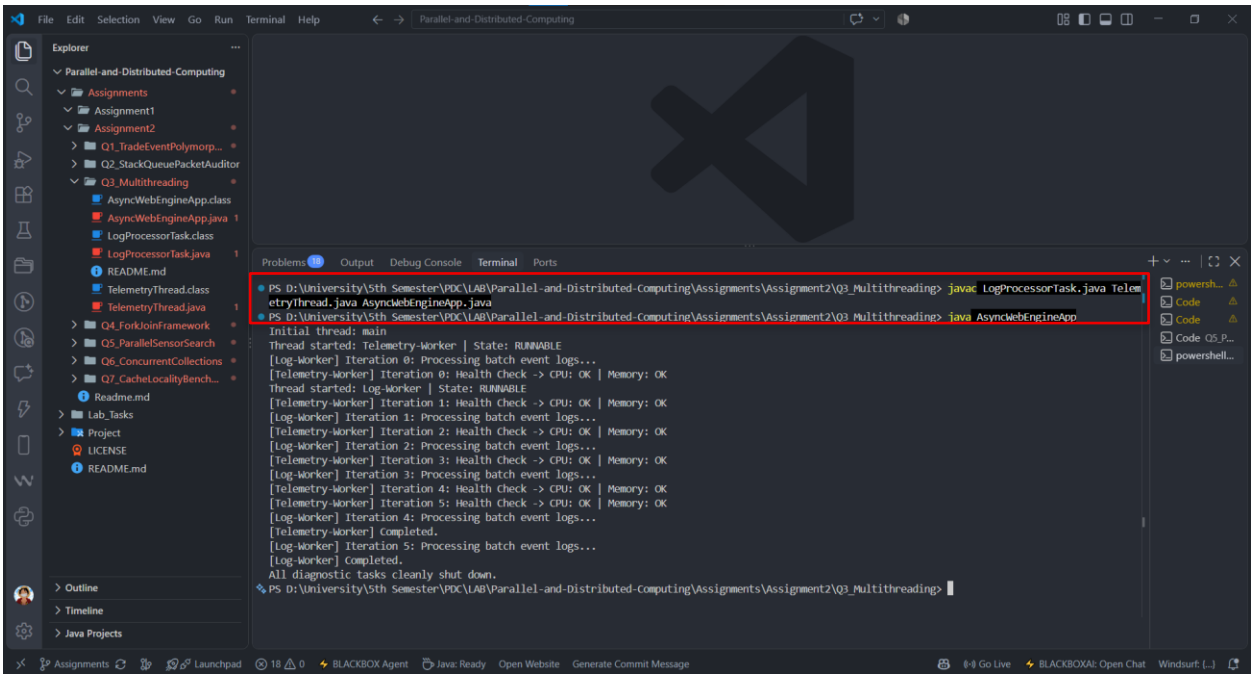
        telemetry.start();
        logWorker.start();

        System.out.println("Thread started: " + telemetry.getName() + " | State:
" + telemetry.getState());
        System.out.println("Thread started: " + logWorker.getName() + " | State:
" + logWorker.getState());
```

```
        try {
            telemetry.join();
            logWorker.join();
        } catch (InterruptedException e) {
            System.out.println("Interrupted Main thread.");
        }

        System.out.println("All diagnostic tasks cleanly shut down.");
    }
}
```

Execution/Compilation Commands:



Output:

```
PS D:\University\5th Semester\PDC\LAB\Parallel-and-Distributed-Computing\Assignments\Assignment2\Q3_Multithreading> jav
etryThread.java AsyncWebEngineApp.java
PS D:\University\5th Semester\PDC\LAB\Parallel-and-Distributed-Computing\Assignments\Assignment2\Q3_Multithreading> jav
Initial thread: main
Thread started: Telemetry-Worker | State: RUNNABLE
[Log-Worker] Iteration 0: Processing batch event logs...
[Telemetry-Worker] Iteration 0: Health Check -> CPU: OK | Memory: OK
Thread started: Log-Worker | State: RUNNABLE
[Telemetry-Worker] Iteration 1: Health Check -> CPU: OK | Memory: OK
[Log-Worker] Iteration 1: Processing batch event logs...
[Telemetry-Worker] Iteration 2: Health Check -> CPU: OK | Memory: OK
[Log-Worker] Iteration 2: Processing batch event logs...
[Telemetry-Worker] Iteration 3: Health Check -> CPU: OK | Memory: OK
[Log-Worker] Iteration 3: Processing batch event logs...
[Telemetry-Worker] Iteration 4: Health Check -> CPU: OK | Memory: OK
[Telemetry-Worker] Iteration 5: Health Check -> CPU: OK | Memory: OK
[Log-Worker] Iteration 4: Processing batch event logs...
[Telemetry-Worker] Completed.
[Log-Worker] Iteration 5: Processing batch event logs...
[Log-Worker] Completed.
All diagnostic tasks cleanly shut down.
PS D:\University\5th Semester\PDC\LAB\Parallel-and-Distributed-Computing\Assignments\Assignment2\Q3_Multithreading>
```

Explanation:

First, I created the LogProcessorTask class by implementing the Runnable interface and overriding its run() method to process log events five times, with a 1000-millisecond pause after each iteration. Next, I created the TelemetryThread class by extending the Thread class and overriding its run() method to display CPU and memory health status five times, with an 800-millisecond pause between iterations. In AsyncWebEngineApp, I displayed the default JVM thread name, created both worker threads with custom names, and started them using the start() method. The program also displays their execution states and uses join() to wait for both threads to finish before printing the final shutdown message. This demonstrates multithreading in Java using both Runnable and Thread, allowing log processing and system health checks to run concurrently.

Question 4: Calculate portfolio risk using Java's Fork/Join framework.

Code:

ParallelRiskCalculator.java

```java
import java.util.concurrent.RecursiveTask;

public class ParallelRiskCalculator extends RecursiveTask<Double> {
    private static final int THRESHOLD = 1000;
    private double[] assets;
    private int start;
    private int end;
```

```
    public ParallelRiskCalculator(double[] assets, int start, int end) {
        this.assets = assets;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Double compute() {
        if ((end - start) <= THRESHOLD) {
            double sum = 0.0;
            for (int i = start; i < end; i++) {
                sum += assets[i];
            }
            return sum;
        } else {
            int mid = (start + end) / 2;
            ParallelRiskCalculator leftTask = new ParallelRiskCalculator(assets,
start, mid);
            ParallelRiskCalculator rightTask = new ParallelRiskCalculator(assets,
mid, end);

            leftTask.fork();
            double rightResult = rightTask.compute();
            double leftResult = leftTask.join();

            return leftResult + rightResult;
        }
    }
}
```

PortfolioThresholdTask.java

```
import java.util.concurrent.RecursiveAction;

public class PortfolioThresholdTask extends RecursiveAction {
    private static final int THRESHOLD = 1000;
    private double[] assets;
    private int start;
    private int end;
    private double riskCap;

    public PortfolioThresholdTask(double[] assets, int start, int end, double
riskCap) {
        this.assets = assets;
```

```
        this.start = start;
        this.end = end;
        this.riskCap = riskCap;
    }

    @Override
    protected void compute() {
        if ((end - start) <= THRESHOLD) {
            for (int i = start; i < end; i++) {
                if (assets[i] > riskCap) {
                    assets[i] = assets[i] * 1.05;
                }
            }
        } else {
            int mid = (start + end) / 2;
            PortfolioThresholdTask left = new PortfolioThresholdTask(assets,
start, mid, riskCap);
            PortfolioThresholdTask right = new PortfolioThresholdTask(assets,
mid, end, riskCap);
            invokeAll(left, right);
        }
    }
}
```

ForkJoinPortfolioApp.java

```
import java.util.concurrent.ForkJoinPool;
import java.util.Random;

public class ForkJoinPortfolioApp {
    public static void main(String[] args) {
        int size = 10000;
        double[] assets = new double[size];
        Random rand = new Random(42);

        for (int i = 0; i < size; i++) {
            assets[i] = 100.0 + (rand.nextDouble() * 900.0);
        }

        ForkJoinPool pool = new ForkJoinPool();

        long startCalc = System.nanoTime();
        ParallelRiskCalculator initialCalc = new ParallelRiskCalculator(assets,
0, size);
        double totalRiskBefore = pool.invoke(initialCalc);
```

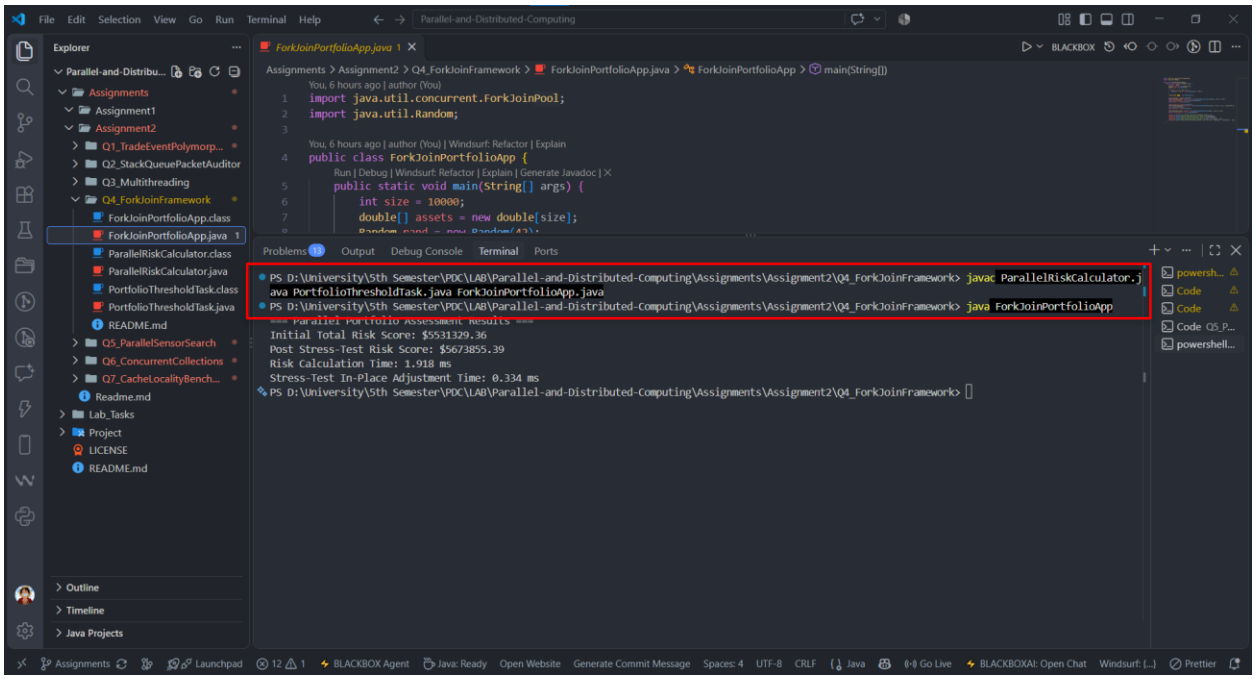
```
        long endCalc = System.nanoTime();

        long startAction = System.nanoTime();
        PortfolioThresholdTask stressTest = new PortfolioThresholdTask(assets, 0,
size, 700.0);
        pool.invoke(stressTest);
        long endAction = System.nanoTime();

        ParallelRiskCalculator postCalc = new ParallelRiskCalculator(assets, 0,
size);
        double totalRiskAfter = pool.invoke(postCalc);

        System.out.println("=== Parallel Portfolio Assessment Results ===");
        System.out.printf("Initial Total Risk Score: $%.2f%n", totalRiskBefore);
        System.out.printf("Post Stress-Test Risk Score: $%.2f%n",
totalRiskAfter);
        System.out.printf("Risk Calculation Time: %.3f ms%n", (endCalc -
startCalc) / 1e6);
        System.out.printf("Stress-Test In-Place Adjustment Time: %.3f ms%n",
(endAction - startAction) / 1e6);
    }
}
```

Execution/Compilation Commands:



Output:

```
PS D:\University\5th Semester\PDC\LAB\Parallel-and-Distributed-Computing\Assignments
ava PortfolioThresholdTask.java ForkJoinPortfolioApp.java
PS D:\University\5th Semester\PDC\LAB\Parallel-and-Distributed-Computing\Assignments
=== Parallel Portfolio Assessment Results ===
Initial Total Risk Score: $5531329.36
Post Stress-Test Risk Score: $5673855.39
Risk Calculation Time: 1.918 ms
Stress-Test In-Place Adjustment Time: 0.334 ms
PS D:\University\5th Semester\PDC\LAB\Parallel-and-Distributed-Computing\Assignments
```

Explanation:

First, I created the ParallelRiskCalculator class by extending RecursiveTask<Double> to calculate the total risk score of an asset array. If the number of elements in a segment is 1000 or less, the class calculates the sum directly; otherwise, it divides the segment into two smaller tasks using the Fork/Join approach and combines their results using fork() and join(). Next, I created the PortfolioThresholdTask class by extending RecursiveAction to apply a 5% increase to asset values greater than the specified risk cap of $700. It also divides larger segments into smaller tasks until they can be processed directly. In ForkJoinPortfolioApp, I generated an array of 10,000 asset values, created a ForkJoinPool, and calculated the initial total risk score. I then applied the stress-test adjustment, recalculated the risk score, and measured the execution time of the initial calculation and adjustment. This demonstrates how Java's Fork/Join framework can divide large tasks into smaller parts and process them in parallel.

Question 5: Find minimum and maximum sensor values using parallel threads.

Code:

SensorSearchEngine.java

```java
public abstract class SensorSearchEngine implements Runnable {
    protected double[] sensorData;
    protected int startInd;
    protected int endInd;

    public SensorSearchEngine(double[] sensorData, int startInd, int endInd) {
        this.sensorData = sensorData;
        this.startInd = startInd;
        this.endInd = endInd;
    }

    public abstract void processGrid();
```

```
    @Override
    public void run() {
        processGrid();
    }
}
```

ParallelMinSearch.java

```
public class ParallelMinSearch extends SensorSearchEngine {
    private double minVal = Double.MAX_VALUE;

    public ParallelMinSearch(double[] sensorData, int startIdx, int endIdx) {
        super(sensorData, startIdx, endIdx);
    }

    @Override
    public void processGrid() {
        for (int i = startInd; i < endInd; i++) {
            if (sensorData[i] < minVal) {
                minVal = sensorData[i];
            }
        }
    }

    public double getMinVal() {
        return minVal;
    }
}
```

ParallelMaxSearch.java

```
public class ParallelMaxSearch extends SensorSearchEngine {
    private double maxVal = Double.MIN_VALUE;

    public ParallelMaxSearch(double[] sensorData, int startIdx, int endIdx) {
        super(sensorData, startIdx, endIdx);
    }

    @Override
    public void processGrid() {
        for (int i = startInd; i < endInd; i++) {
            if (sensorData[i] > maxVal) {
                maxVal = sensorData[i];
            }
```

```
        }
    }

    public double getMaxVal() {
        return maxVal;
    }
}
```

SensorGridMonitorApp.java

```
import java.util.Random;

public class SensorGridMonitorApp {
    public static void main(String[] args) throws InterruptedException {
        int dataSize = 50000;
        double[] telemetry = new double[dataSize];
        Random rand = new Random(100);

        for (int i = 0; i < dataSize; i++) {
            telemetry[i] = 10.0 + (rand.nextDouble() * 90.0);
        }

        int numWorkers = 4;
        int chunkSize = dataSize / numWorkers;

        ParallelMinSearch[] minTasks = new ParallelMinSearch[numWorkers];
        ParallelMaxSearch[] maxTasks = new ParallelMaxSearch[numWorkers];
        Thread[] minThreads = new Thread[numWorkers];
        Thread[] maxThreads = new Thread[numWorkers];

        long startTime = System.nanoTime();

        for (int i = 0; i < numWorkers; i++) {
            int start = i * chunkSize;
            int end = (i == numWorkers - 1) ? dataSize : start + chunkSize;

            minTasks[i] = new ParallelMinSearch(telemetry, start, end);
            maxTasks[i] = new ParallelMaxSearch(telemetry, start, end);

            minThreads[i] = new Thread(minTasks[i]);
            maxThreads[i] = new Thread(maxTasks[i]);

            minThreads[i].start();
            maxThreads[i].start();
        }
```

```
        for (int i = 0; i < numWorkers; i++) {
            minThreads[i].join();
            maxThreads[i].join();
        }

        double globalMin = Double.MAX_VALUE;
        double globalMax = Double.MIN_VALUE;

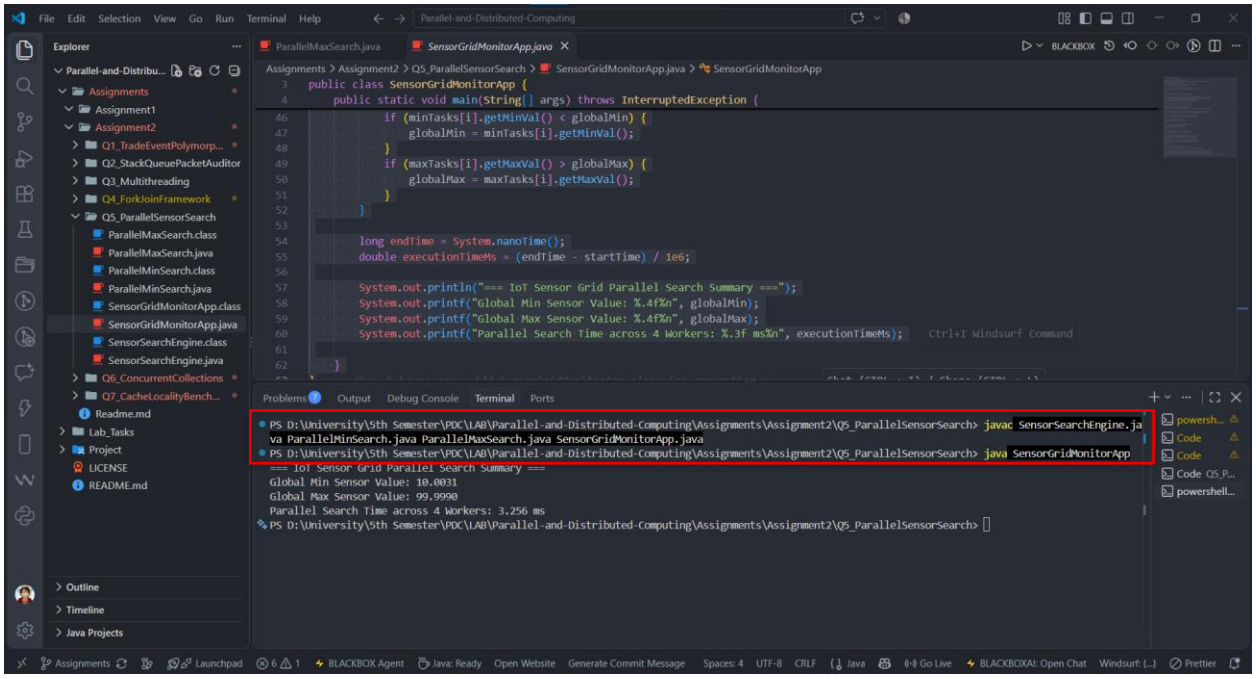
        for (int i = 0; i < numWorkers; i++) {
            if (minTasks[i].getMinVal() < globalMin) {
                globalMin = minTasks[i].getMinVal();
            }
            if (maxTasks[i].getMaxVal() > globalMax) {
                globalMax = maxTasks[i].getMaxVal();
            }
        }

        long endTime = System.nanoTime();
        double executionTimeMs = (endTime - startTime) / 1e6;

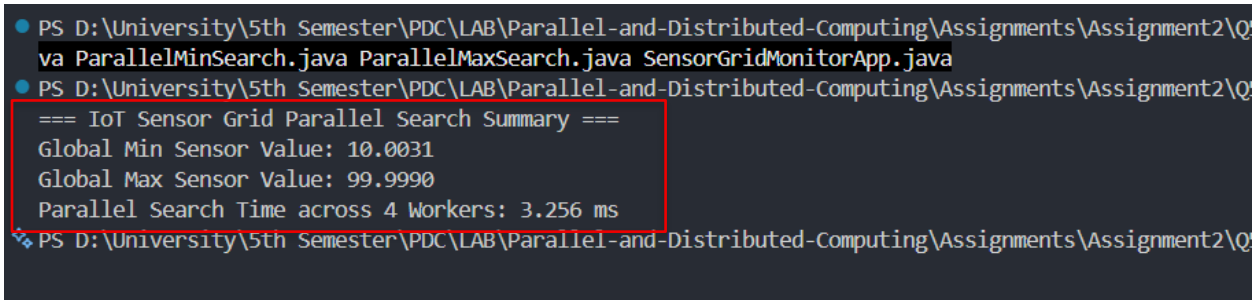
        System.out.println("=== IoT Sensor Grid Parallel Search Summary ===");
        System.out.printf("Global Min Sensor Value: %.4f%n", globalMin);
        System.out.printf("Global Max Sensor Value: %.4f%n", globalMax);
        System.out.printf("Parallel Search Time across 4 Workers: %.3f ms%n",
executionTimeMs);

    }
}
```

Execution/Compilation Commands:



Output:



Explanation:

First, I created the abstract SensorSearchEngine class to store the sensor data array and the starting and ending indexes for each worker. It implements the Runnable interface and defines an abstract processGrid() method. Then, I created the ParallelMinSearch and ParallelMaxSearch classes, which extend the parent class and override processGrid() to find the minimum and maximum values within their assigned array segments. In SensorGridMonitorApp, I generated 50,000 sensor values and divided the array into four equal parts. Separate threads were created for minimum and maximum searches on each part and started concurrently. The program uses join() to wait for all threads to finish before comparing their results to find the global minimum and maximum values. Finally, it measures and displays the total execution time using System.nanoTime(). This demonstrates multithreading, inheritance, method overriding, and parallel searching across a large sensor dataset.

Question 6: **Implement a Java inventory catalog concurrency benchmark comparing Collections.synchronizedMap() and ConcurrentHashMap in terms of execution time and throughput.**

Code:

InventoryCatalogManager.java

```
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class InventoryCatalogManager {
    private static final int NUM_THREADS = 10;
    private static final int OPS_PER_THREAD = 50000;
    private static final int TOTAL_OPS = NUM_THREADS * OPS_PER_THREAD;

    public static void main(String[] args) throws InterruptedException {
        System.out.println("=== Inventory Catalog Concurrency Benchmark ===\n");
        benchmarkSynchronizedMap();
        System.out.println();
        benchmarkConcurrentMap();
    }

    public static void benchmarkSynchronizedMap() throws InterruptedException {
        Map<String, Integer> syncMap = Collections.synchronizedMap(new
HashMap<>());

        for (int i = 0; i < 100; i++) {
            syncMap.put("PROD_" + i, 1000);
        }

        Thread[] workers = new Thread[NUM_THREADS];
        long startTime = System.currentTimeMillis();

        for (int t = 0; t < NUM_THREADS; t++) {
            workers[t] = new Thread(() -> {
                for (int i = 0; i < OPS_PER_THREAD; i++) {
                    String key = "PROD_" + (i % 100);

                    syncMap.put(key, syncMap.get(key) + 1);
                }
            });
            workers[t].start();
```

```
        }

        for (Thread worker : workers) {
            worker.join();
        }

        long totalTime = System.currentTimeMillis() - startTime;
        double throughput = (TOTAL_OPS / (totalTime / 1000.0));

        System.out.println("--- Collections.synchronizedMap() ---");
        System.out.println("Total Execution Time: " + totalTime + " ms");
        System.out.printf("Throughput: %.2f ops/sec%n", throughput);
    }

    public static void benchmarkConcurrentMap() throws InterruptedException {
        Map<String, Integer> concurrentMap = new ConcurrentHashMap<>();

        for (int i = 0; i < 100; i++) {
            concurrentMap.put("PROD_" + i, 1000);
        }

        Thread[] workers = new Thread[NUM_THREADS];
        long startTime = System.currentTimeMillis();

        for (int t = 0; t < NUM_THREADS; t++) {
            workers[t] = new Thread(() -> {
                for (int i = 0; i < OPS_PER_THREAD; i++) {
                    String key = "PROD_" + (i % 100);

                    concurrentMap.compute(key, (k, v) -> (v == null ? 1 : v +
1));
                }
            });
            workers[t].start();
        }

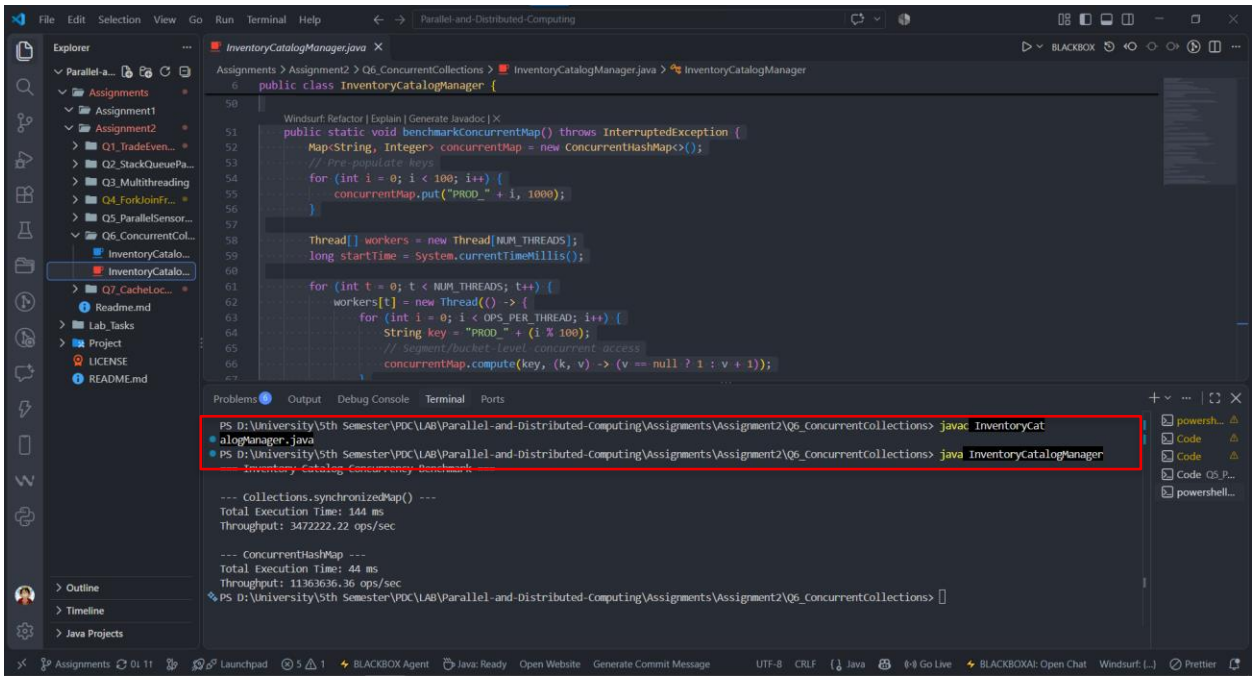
        for (Thread worker : workers) {
            worker.join();
        }

        long totalTime = System.currentTimeMillis() - startTime;
        double throughput = (TOTAL_OPS / (totalTime / 1000.0));

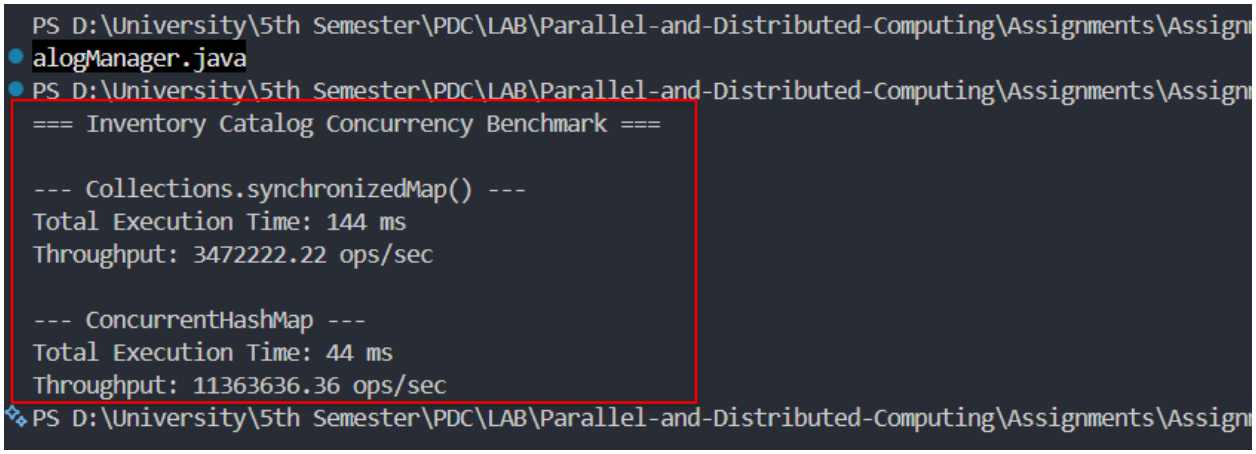
        System.out.println("--- ConcurrentHashMap ---");
        System.out.println("Total Execution Time: " + totalTime + " ms");
```

```
        System.out.printf("Throughput: %.2f ops/sec%n", throughput);
    }
}
```

Execution/Compilation Commands:



Output:



Explanation:

First, I created the InventoryCatalogManager class to compare the performance of Collections.synchronizedMap() and ConcurrentHashMap when multiple threads access and update product stock. Both maps are initialized with 100 product IDs, each having an initial stock value of 1000. In each

benchmarking method, I created 10 worker threads, with each thread performing 50,000 stock lookup and update operations. The synchronized map uses get() and put() to read and update values, while the concurrent map uses compute() to update each value. The join() method ensures that all worker threads finish before the total execution time is calculated. Finally, the program calculates and displays the execution time and throughput in operations per second for both maps. This allows their performance to be compared under concurrent workloads and demonstrates Java's thread-safe collections.

Question 7: Develop a Java cache locality benchmark to compare row-major and column-major traversal performance of a 2000 × 2000 matrix over 10 iterations.

Code:

CacheLocalityBenchmark.java

```
public class CacheLocalityBenchmark {
    private static final int N = 2000;
    private static final int ITERATIONS = 10;
    private static double[][] matrix = new double[N][N];

     public static void main(String[] args) {
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                matrix[i][j] = 1.0;
            }
        }

        System.out.println("=== Cache Locality Traversal Benchmark (2000 x 2000
Matrix) ===");

        traverseRowMajor();
        traverseColumnMajor();

        long startRow = System.nanoTime();
        for (int it = 0; it < ITERATIONS; it++) {
            traverseRowMajor();
        }
        long endRow = System.nanoTime();
        double rowTotalMs = (endRow - startRow) / 1e6;
        double rowAvgMs = rowTotalMs / ITERATIONS;

        long startCol = System.nanoTime();
        for (int it = 0; it < ITERATIONS; it++) {
            traverseColumnMajor();
        }
```

```
        long endCol = System.nanoTime();
        double colTotalMs = (endCol - startCol) / 1e6;
        double colAvgMs = colTotalMs / ITERATIONS;

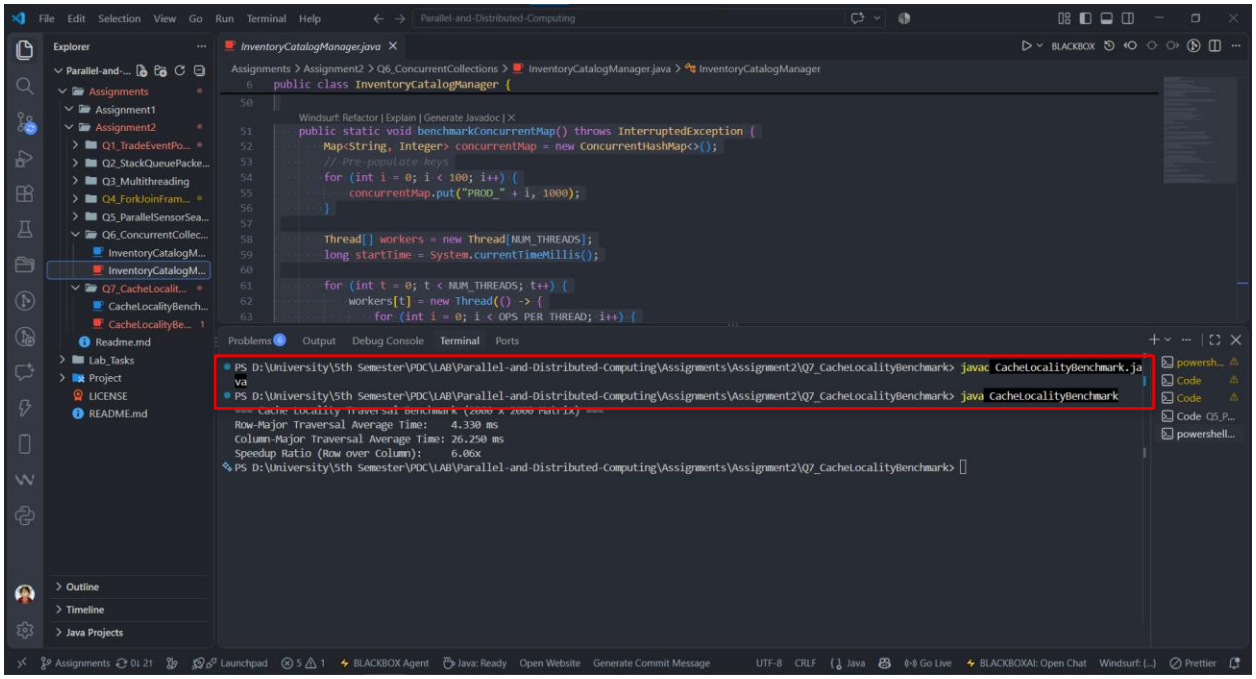
        double speedup = colAvgMs / rowAvgMs;

        System.out.printf("Row-Major Traversal Average Time:    %.3f ms%n",
rowAvgMs);
        System.out.printf("Column-Major Traversal Average Time: %.3f ms%n",
colAvgMs);
        System.out.printf("Speedup Ratio (Row over Column):     %.2fx%n",
speedup);
     }

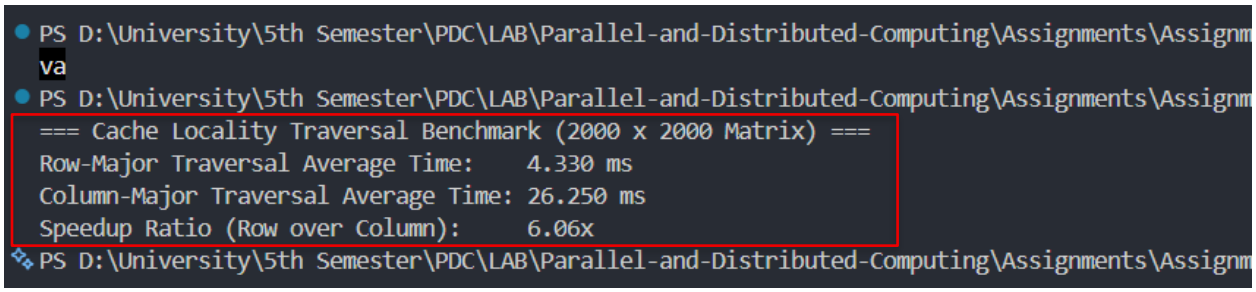
     public static double traverseRowMajor() {
        double sum = 0.0;
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                sum += matrix[i][j];
            }
        }
        return sum;
    }

    public static double traverseColumnMajor() {
        double sum = 0.0;
        for (int j = 0; j < N; j++) {
            for (int i = 0; i < N; i++) {
                sum += matrix[i][j];
            }
        }
        return sum;
    }
}
```

Execution/Compilation Commands:



Output:



Explanation:

First, I created the CacheLocalityBenchmark class with a 2000 × 2000 matrix and initialized all its elements to 1.0. I implemented two methods to compare different ways of accessing the matrix. The traverseRowMajor() method uses nested loops to access elements row by row, while traverseColumnMajor() accesses them column by column by switching the loop order. Before measuring the execution time, I ran both methods once to warm up the JVM. Then, I used System.nanoTime() to measure the time taken by each method over 10 iterations and calculated the average execution time. Finally, the program calculated the speedup ratio by dividing the average column-major time by the average row-major time and displayed the results in milliseconds. This benchmark demonstrates how memory access patterns and CPU cache locality can affect program performance.

~ End ~